

Threads (I)

Lezione 16

Sistemi Operativi - II Modulo
(Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

- Concetto di thread e differenza con i processi
- Modelli di applicazioni multi-thread
- Le librerie
- La libreria pthread (POSIX thread)
 - Esempi
 - Esercizi
- Implementazione

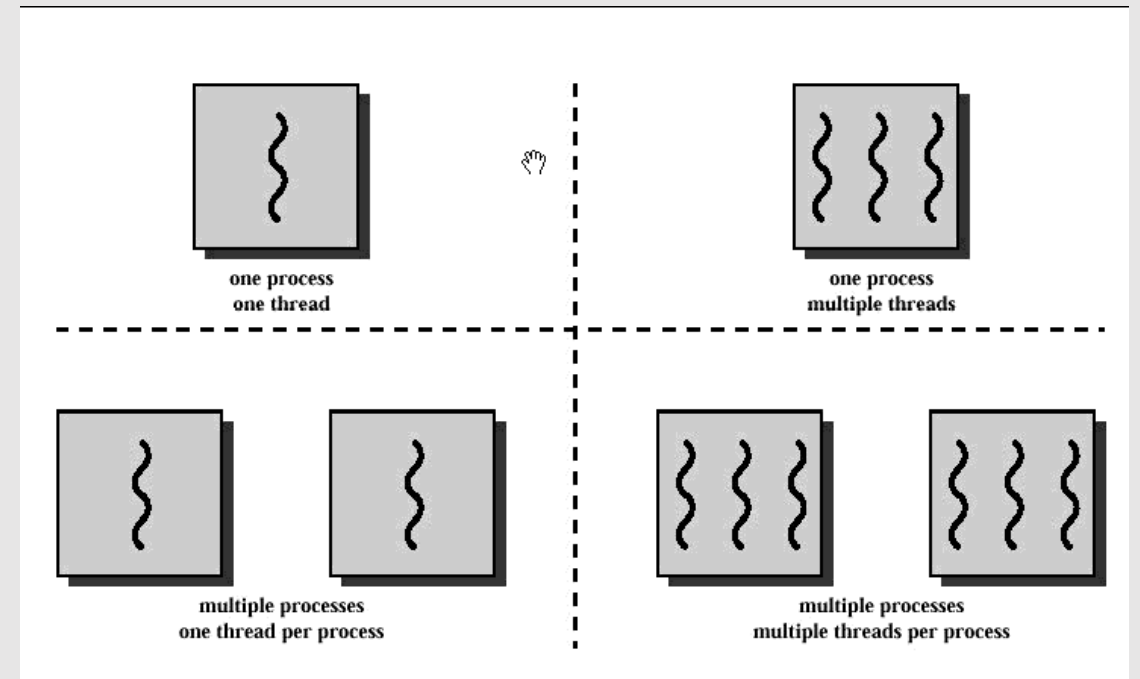
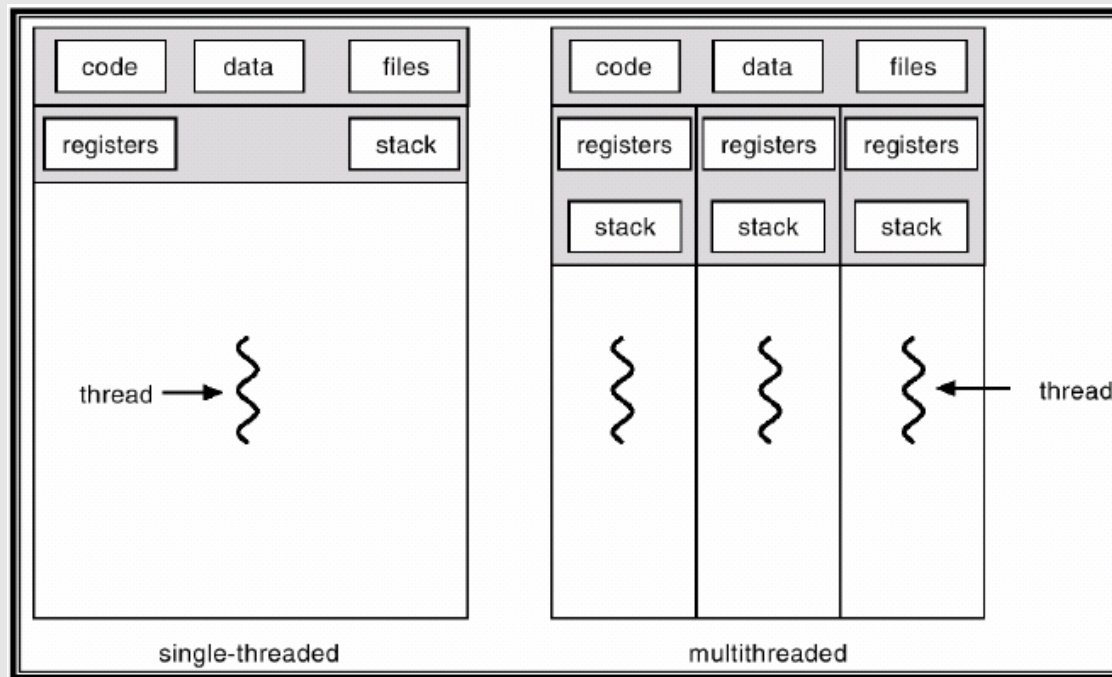


Applicazioni multithread

- In una applicazione tradizionale, il programmatore definisce un unico flusso di esecuzione delle istruzioni
 - La CPU esegue istruzioni macchina in sequenza il flusso di esecuzione “segue” la logica del programma (cicli, funzioni, chiamate di sistema, gestori di segnali. . .)
 - Quando il flusso di esecuzione arriva ad eseguire una `exit()` l'applicazione termina
- Le applicazioni *multithread* consentono al programmatore di definire diversi flussi (o «fili», «trame» (come in un tessuto)) di esecuzione
 - Ciascun flusso di esecuzione condivide le strutture di dati principali dell'applicazione
 - Ciascun flusso di esecuzione procede in modo concorrente ed indipendente dagli altri flussi
 - L'applicazione finisce solo quando tutti i flussi di esecuzione vengono terminati



Processi vs. Threads



Motivazioni: Elevato parallelismo interno dei calcolatori elettronici

- DMA (Direct Memory Access)
 - trasferimento dati tra memoria primaria e periferiche di I/O senza intervento della CPU
- Hyperthreading
 - supporto a diversi flussi di esecuzione, ciascuno con un proprio insieme di registri, che si alternano sulle unità funzionali della CPU
- Multicore
 - diversi core di calcolo integrati sullo stesso chip e che condividono alcune risorse hardware quali cache di 2° livello, MMU, ...
- Multiprocessori
 - diverse CPU integrate sulla stessa scheda madre

È difficile scrivere applicazioni tradizionali (unico flusso) che sfruttino a fondo il parallelismo interno del calcolatore

Esempio di applicazione multithread

- Un browser web potrebbe essere costituito dai seguenti thread
 - Thread principale di controllo dell'applicazione
 - Thread per l'interazione con l'utente
 - Thread per la visualizzazione (rendering) delle pagine in formato HTML
 - Thread per la gestione dei trasferimenti di pagine e file dalla rete
 - Thread per l'esecuzione dei frammenti di script integrati nelle pagine Web
 - Thread per l'esecuzione dei programmi Java, Flash, ecc.

Ciascun thread
compie il proprio
lavoro eseguendo
un flusso di
istruzioni
indipendente e
cooperando con gli
altri thread

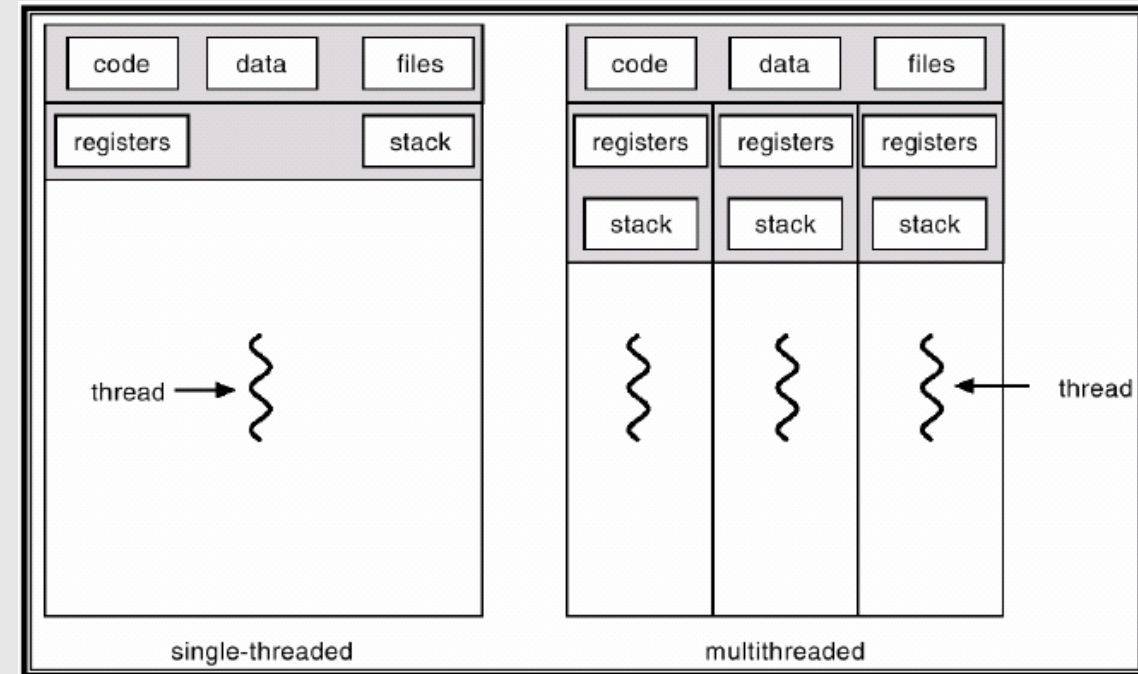
Vantaggi del multithreading

- Riduzione del tempo di risposta
 - anche se una parte dell'applicazione è bloccata in attesa di eventi esterni, un altro thread può essere eseguito per interagire con l'utente o gestire altri eventi
- Migliore condivisione delle risorse
 - tutti i thread di una applicazione condividono le risorse (strutture di dati in memoria, file aperti), e la comunicazione tra i thread è immediata
- Maggiore efficienza
 - rispetto ad una applicazione costituita da più processi cooperanti, l'applicazione multithread è più efficiente, perché il SO gestisce i thread più rapidamente
 - In Linux, creare un thread richiede 1/10 del tempo richiesto per la creazione di un processo
- Maggiore scalabilità
 - i thread possono sfruttare in modo implicito il parallelismo interno del calcolatore



Processi e threads

- Un processo per una applicazione monothread è costituito da:
 - Codice (istruzioni macchina in memoria)
 - Strutture di dati (variabili globali in memoria, heap)
 - File aperti
 - Contenuto dei registri della CPU (contesto)
 - Posizione e contenuto dello stack UM
- In una applicazione multithread alcune risorse sono comuni e condivise tra tutti i thread:
 - Codice e Strutture di dati
 - File aperti
- Altre risorse sono private per ciascun thread:
 - Contenuto dei registri della CPU (contesto)
 - Posizione e contenuto dello stack



Processi e threads (cont.)

Processi	Thread
Istanze di un programma in esecuzione (heavyweight)	Componente di un processo e più piccola unità di esecuzione (lightweight)
Cambio di contesto (context switching) richiede interazione con SO	Context Switch non richiede interazione con SO
Ogni processo ha il suo spazio di memoria	Usano la memoria del processo a cui appartengono
Richiedono più risorse di sistema	Richiedono meno risorse di sistema
Difficili da creare	Facili da creare
Comunicazione tra processi lenta in quanto ogni processo ha un differente indirizzo di memoria	Comunicazione tra thread è veloce in quanto i thread condividono lo stesso indirizzo (e area) di memoria del processo a cui appartengono
Ogni processo esegue indipendentemente (isolato)	Ogni thread può leggere, scrivere, modificare dati di altri thread

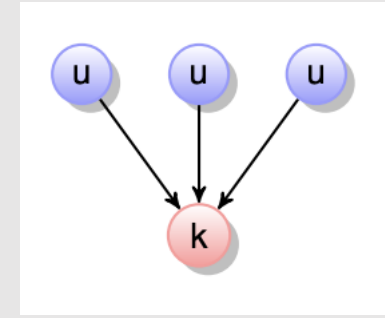
Implementazione di applicazioni multithread

- Esistono principalmente due componenti in un implementazione delle applicazioni multithread
 - Implementazione a livello utente
 - Implementazione a livello kernel
- In sostanza tutte le implementazioni sono caratterizzate dalla relazione tra:
 - Thread utente
 - il flusso di esecuzione dell'applicazione
 - Thread kernel
 - l'astrazione (strutture di dati, servizi) definita all'interno del nucleo del SO per gestire un flusso di esecuzione

La definizione di thread utente o thread kernel non è basata su una diversa modalità d'esecuzione (User Mode o Kernel Mode)

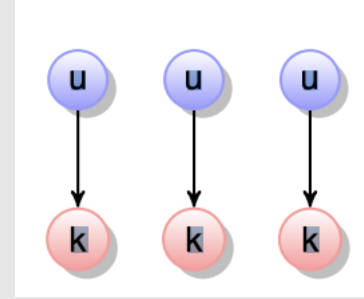


Modello “da molti a 1”



- Un'applicazione multithread è costituita da un singolo processo
- A diversi thread utente corrisponde un singolo thread kernel
 - Un singolo processo tradizionale
- Il kernel non è coinvolto nella gestione dei flussi dell'applicazione
- L'applicazione (eventualmente usando una libreria di sistema) gestisce autonomamente i thread utente:
 - Organizza i vari flussi di esecuzione
 - Salva e ripristina gli stack UM e i contesti
- Se un thread invoca una chiamata di sistema bloccante, il processo (e quindi tutti i thread utente) vengono bloccati
- Impossibile sfruttare in modo implicito il parallelismo interno del calcolatore
 - Implementazione detta “**a livello utente**” o green threads

Modello “da 1 a 1”



- Ciascun thread utente della applicazione corrisponde ad un singolo thread kernel
- Il kernel si occupa della gestione e schedulazione dei thread kernel
 - Perciò gestisce anche i thread utente
- L'applicazione utilizza le API definite in una libreria di sistema:
 - Crea e distrugge i thread utente
 - Gestisce comunicazione e sincronizzazione
- La libreria implementa i servizi richiesti dall'applicazione invocando opportune chiamate di sistema
- Ciascun thread utente può invocare chiamate di sistema bloccanti senza bloccare gli altri thread
- L'applicazione sfrutta in modo implicito il parallelismo interno del calcolatore
 - Implementazione detta “a livello kernel” o *native threads*

Implementazione dei threads

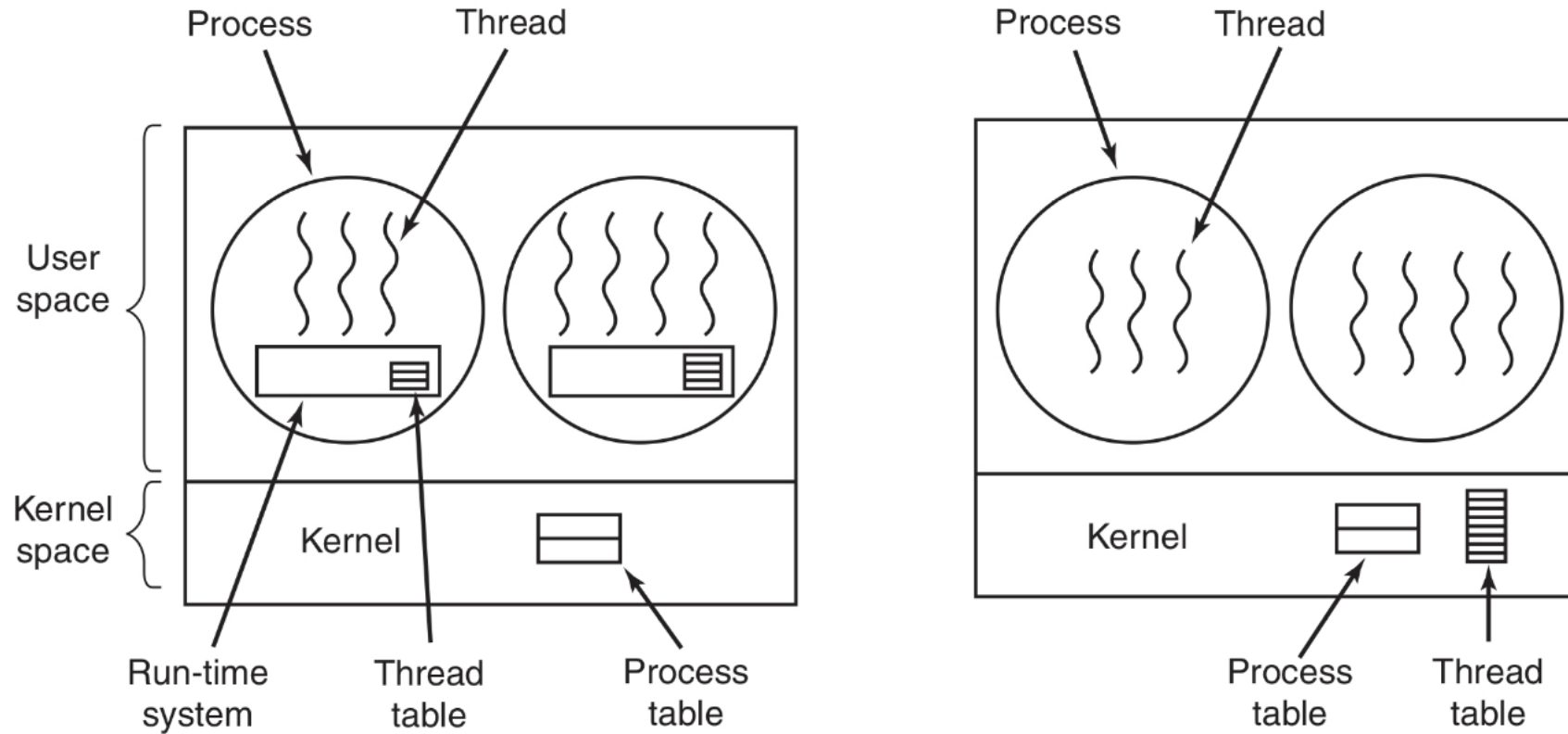


Figure 2-16. (a) A user-level threads package. (b) A threads package managed by the kernel.

User-level vs. Kernel-level threads

User-level

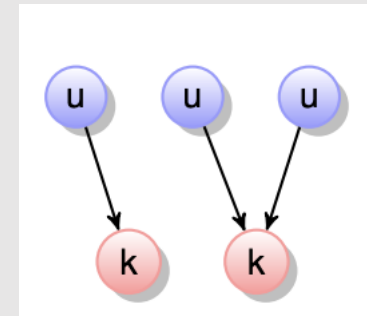
- SO ignaro dei threads
- Facile implementazione
- Cambio contesto veloce; supporto HW non necessario
- Una chiamata bloccante di un thread blocca l'intero processo
- Esempi: Posix & Java threads

Kernel-level

- SO consapevole dei threads
- Implementazione complicata
- Cambio contesto consuma tempo e risorse, e richiede supporto
- Una chiamata bloccante di un thread NON blocca l'intero processo
- Esempi: Windows, Solaris



Modello da molti a molti



- n_u thread utente dell'applicazione corrispondono a n_k thread kernel ($n_k \leq n_u$)
- Il kernel del SO si occupa della gestione e schedulazione dei thread kernel
 - Non gestisce direttamente i thread utente
- L'applicazione utilizza le API definite in una libreria di sistema:
 - Definisce il numero n_k di thread kernel
 - Crea e distrugge i thread utente
 - Mappa i thread utente sui thread kernel
- La libreria di sistema:
 - Usa chiamate di sistema per gestire i thread kernel
 - Gestisce la schedulazione e lo stack UM dei thread utente mappati sullo stesso thread kernel
 - Gestisce comunicazione e sincronizzazione

Esempi di implementazione

- Esempi di implementazioni con modello “da molti a uno”
 - La libreria green threads disponibile in Sun (Oracle) Solaris
 - La libreria GNU Pth (GNU Portable Threads)
 - Le librerie che implementano i thread a livello utente possono essere utilizzate con qualunque sistema operativo
- Tutti i maggiori SO oggi supportano i thread nativi, e dunque i modelli “da uno a uno” e “da molti a molti”
 - Linux, Windows, MacOS X tendono ad adottare il modello “da uno a uno”
 - IRIX, HP-UX, Tru64 UNIX adottano il modello “da molti a molti”
- In ogni caso la scelta del modello “da uno a uno” piuttosto che “da molti a molti” dipende dalla libreria di thread utilizzata, e non dal sistema operativo

Librerie dei thread

- Generalmente il programmatore utilizza una libreria di sistema per realizzare una applicazione multithread
- Le API offerte dalla libreria non sono direttamente correlate con la tipologia di thread utilizzata
 - Possono esistere diverse versioni di una libreria con identiche API ma thread a livello utente oppure kernel
 - Esempio: la libreria pthreads (POSIX threads)
- Alcune librerie e relative API sono specifiche per un determinato sistema operativo e tipologia di thread
 - Esempio: la libreria per i thread delle API Win32
- Alcune librerie di thread e relative API sono specifiche di un linguaggio ad alto livello
 - L'uso della libreria è implicito e automatico
 - La libreria utilizza una libreria di thread di livello più basso
 - Esempio: la libreria di thread del linguaggio Java



Libreria Pthreads

- La libreria pthreads è definita dallo standard POSIX (IEEE 1003.1c)
- Lo standard definisce le API, ma non stabilisce quale debba essere la loro implementazione in uno specifico SO
- In Linux sono coesistite tre diverse implementazioni:
 - LinuxThreads: prima implementazione, basata sul modello “da uno a uno”, non più supportata
 - NGPT (Next Generation POSIX Threads): sviluppata da IBM, sul modello “da molti a molti”, ora non più supportata
 - NPTL (Native POSIX Threads Library): ultima implementazione, più efficiente, più aderente allo standard, sul modello “da uno a uno”
- Oggi in Linux si utilizza esclusivamente NPTL



Implementazione



Creazione di un nuovo thread in pthreads

- La funzione di libreria `pthread_create()` crea un nuovo thread:
 - `int pthread_create(pthread_t *ptid, pthread_attr_t *pattr, void *(*start)(void *), void *arg)`
- `ptid`: puntatore a variabile di tipo `pthread_t` che conterrà l'identificatore del nuovo thread (TID)
- `pattr`: puntatore ad una variabile contenente attributi (flag) per la creazione del thread
- `start`: funzione inizialmente eseguita dal thread, con prototipo:
`void *start(void *);`
- `arg`: puntatore passato come argomento a `start()`

Terminazione di un nuovo thread in pthreads

- La funzione di libreria `pthread_exit()` termina l'esecuzione del thread che la invoca:
 - `void pthread_exit(void *value_ptr);`
- Il valore passato come argomento può essere passato ad altri thread dello stesso processo (`join`)
- La funzione viene implicitamente invocata quando la funzione iniziale `start` del thread termina
- Se viene eseguita dall'ultimo thread di un processo, il processo stesso termina con una `exit(0)`



Attesa della terminazione di un thread in pthreads

- La funzione di libreria `pthread_join()` attende la conclusione di un thread:
 - `int pthread_join(pthread_t tid, void *pret);`
- `tid` è l'identificatore del thread di cui si vuole attendere la terminazione
- `pret` è un puntatore ad una variabile che riceverà il valore passato dal thread terminato in `pthread_exit()`
- Non esiste modo di indicare che si vuole attendere la terminazione di un thread qualunque

Esempio 1

```
#include <pthread.h>
long start(long v)
{
    return v+1;
}
int main(int argc, char *argv[])
{
    pthread_t tid;
    pthread_create(&tid, 0, start, argc);
    pthread_join(tid, &argc);
    return argc;
}
```

- file `pthreadHW.c`
- compilare con gcc indicando l'opzione `-pthread`



Terminazione di un processo multithread

- In Unix un processo viene terminato con `exit()` :
 - `void exit(int status);`
- In **Linux** le cose sono più complicate:
 - **Premessa:** in Linux e Unix le chiamate di sistema sono ‘confezionate’ (o esposte) da una funzione C cosiddetta ***wrapper***. Di solito (ma **non** sempre!) la wrapper ha lo stesso nome della chiamata di sistema.
 - La *chiamata di sistema* `_exit` termina un singolo thread
 - La *chiamata di sistema* `exit_group` termina tutti i thread di un processo
 - La funzione *wrapper* `_exit()` esegue la chiamata di sistema `exit_group`, non `_exit` [Per invocare la chiamata di sistema `_exit` si deve usare la chiamata di sistema `syscall` – vedere `man 2 syscall`.]

Terminazione di un processo multithread

- (cont.) In **Linux** le cose sono più complicate:
 - La *funzione di libreria* `exit()` invoca, alla fine, la wrapper `_exit()`
 - Eseguire `return` in `main()` è equivalente ad invocare la funzione di libreria `exit()`
 - La *funzione wrapper* `_Exit()` (standard C99) è equivalente alla funzione `wrapper _exit()`
 - La *funzione di libreria* **`pthread_exit()`** invoca direttamente la chiamata di sistema `_exit`

Esempio 2: pthread.c

- Oltre all'uso di pthread_create/_join vediamo l'uso di
 - `int pthread_attr_init(pthread_attr_t *attr);`
 - `int pthread_attr_destroy(pthread_attr_t *attr);`
 - `int pthread_attr_setstacksize(pthread_attr_t *attr, size_t stacksize);`
- pthread_attr_t è un tipo di dato (struttura) che contiene gli attributi del thread
 - <https://pubs.opengroup.org/onlinepubs/7908799/xsh/systypes.h.html>
 - <https://pubs.opengroup.org/onlinepubs/7908799/xsh/pthread.h.html>

Principali Attributi Pthread in `pthread_attr_t`

- `scope` `PTHREAD_SCOPE_PROCESS`
 - New thread is unbound; it competes for resources with all the other threads in the process and in the system.
- `detachstate` `PTHREAD_CREATE_JOINABLE`
 - Exit status and thread are preserved after the thread terminates.
- `stackaddr` `NULL`
 - New thread has system-allocated stack address.
- `stacksize` 1 megabyte
 - New thread has system-defined stack size.
- `priority`
 - New thread inherits parent thread priority.
- `inheritsched` `PTHREAD_INHERIT_SCHED`
 - New thread inherits parent thread scheduling priority.
- `schedpolicy` `SCHED_OTHER`
 - New thread uses Solaris-defined fixed priority scheduling; threads run until preempted by a higher-priority thread or until they block or yield.



Per leggere e modificare gli attributi di un pthread

- Famiglia di funzioni `pthread_attr_*`, ad es.:

- `pthread_getattr_np(3)`
- `pthread_attr_setdetachstate(3)`
- `pthread_attr_setinheritsched(3)`
- `pthread_attr_setschedparam(3)` (priority)
- `pthread_attr_setschedpolicy(3)`
- `pthread_attr_setscope(3)`
- `pthread_attr_setstack(3)`
- `pthread_attr_setstackaddr(3)`
- `pthread_attr_setstacksize(3)`

Esercizio

nel file `pthreadAttr.c` c'è un esempio di uso delle funzioni `pthread_attr_*`.

Eseguire il programma, comprenderne il funzionamento e l'uso delle varie funzioni di libreria usate. Basatevi ovviamente sul man (da cui tra l'altro il programma è stato preso)



Esercizi

- Es1: Scrivere un programma C che crei un “Thread” ed attende la terminazione dello stesso. Il thread scriverà a video in 20 secondi i numeri da 0 a 19.
- Es2: Scrivere un programma C che crei 5 “Thread” ciascuno dei quali stampi a video il messaggio “Hello Word sono il Thread #” ed attenda la terminazione dei 5 thread.
- Es3: Scrivere un programma C che crei 5 “Thread” a cui vengono passati tramite una struttura due parametri: il numero del thread ed un messaggio.
- Es4: Scrivere un programma C che accetta un numero intero $< n >$ da riga di comando, crea un Thread che sommerà ad una variabile globale intera di valore m il valore di n ed infine ne stampa il risultato a video.
- Es5: Scrivere un programma C che crei un Thread il quale insieme al programma main incrementa di venti volte una variabile globale (myglobal). Il thread chiama una funzione “incrementa”. Supporremo che l’accesso alla variabile globale non venga regolamentato da alcun meccanismo di gestione della concorrenza.
- Es6: Scrivere un programma C che crea due Thread “somma1” e “somma2”, entrambi accedono alle variabili test.a e test.b di una struttura dati test condivisa incrementandole di 1 per 10 volte, aspettano 2 secondi prima di stampare a video i valori delle due variabili. Supporremo che tale accesso non venga regolamentato da alcun meccanismo di gestione della concorrenza.

Implementazione dei thread in Linux

- L'implementazione dei thread in Linux è basata sul concetto di processo leggero o LWP (Light Weight Process)
- Un processo leggero è un processo che condivide alcune risorse selezionate con il proprio genitore
- Per creare processi leggeri si utilizza l'API `clone()` con diversi possibili flag `CLONE_XXX`, ad esempio
 - `CLONE_FILES` descrittori di file
 - `CLONE_FS` file system (ad es., directory corrente)
 - `CLONE_SIGHAND` gestori dei segnali
 - `CLONE_THREAD` stesso "processo" (gruppo di thread)
 - `CLONE_VM` spazio di memoria
- `fork()` $\equiv$ `clone()` senza alcuno degli attributi precedenti
- `pthread_create()` $\equiv$ `clone()` con tutti gli attributi



clone (3) - funzione di libreria

- La *funzione di libreria* `clone()` è simile alla funzione `pthread_create()`:
 - `int clone(start, stack, flags, arg, ...);`
- `start` è la funzione inizialmente eseguita dal processo leggero
- `stack` è l'indirizzo della cima dello stack UM del nuovo processo leggero
- `flags` sono i flag `CLONE_XXX`
- `arg` è l'argomento passato a `start()`
- La funzione di libreria `clone()` si basa su una chiamata di sistema `clone` con semantica differente

clone (2) - chiamata di sistema

- La chiamata di sistema clone è simile alla fork():
 - `int clone(flags, stack);`
- `flags` sono i flag `CLONE_XXX`
- `stack` è l'indirizzo della cima dello stack UM del nuovo processo
leggero se è NULL, il figlio utilizza una copia dello stack del padre
- L'esecuzione del figlio inizia con l'istruzione seguente l'invocazione di clone

Domande?

